Chats With Legal Application

Course Name: GenAI

Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	VISHAL RATHOD	EN22CS306059
2.	PALAK GUPTA	EN22CS306036
3.	JAY PRAKASH SINGH	EN22CS304031
4.	RAJVEER MUKATI	EN22CS304051
5.	KOMAL JADHAV	EN22CS303028
6.	DIYA GOYAL	EN22CS304025

Group Name: G11D11

Project Number: DO-37

Industry Mentor Name: Mr. Yash Sir

University Mentor Name: Dr. Hemlata Patel

Academic Year: 2025-26

Introduction.

1.1. Scope of the document.

1.2. Intended Audience

1.3. System overview.

2. System Design.

2.1. Application Design

2.2. Process Flow.

2.3. Information Flow.

2.4. Components Design

2.5. Key Design Considerations

2.6. API Catalogue.

3. References

1. Introduction

The **Chat with Legal Documents (RAG)** system is designed to enable users to interact with legal documents using natural language queries. The system uses **Retrieval-Augmented Generation (RAG)** to ensure that responses are accurate and grounded in the uploaded document content.

The system processes legal documents such as contracts, agreements, and reports by extracting text, converting it into embeddings, and storing it in a vector database. When a user asks a question, the system retrieves relevant content and generates a response using a Large Language Model (LLM).

This document provides a **Low Level Design (LLD)** of the system, describing the internal structure, modules, workflows, APIs, and technical implementation details.

1.1 Scope of the Document

This document covers the detailed design of the **Chat with Legal Documents system**, including:

- Internal system architecture
- Module-level design and responsibilities
- Data processing and workflow
- Interaction between components
- API design and structure
- Information flow within the system

It focuses on the **technical implementation details** required to build the system and does not cover business-level decisions or UI design specifics.

1.2 Intended Audience

This document is intended for:

- **Developers** – to understand implementation and coding structure
- **System Designers** – to understand architecture and component interaction
- **Project Evaluators/Faculty** – to assess system design and completeness
- **Testers** – to understand workflows and validation points

---

1.3 System Overview

The system allows users to:

- Upload legal documents (PDFs, reports, agreements)
- Process documents into structured text
- Store document embeddings in a vector database
- Ask questions using natural language
- Receive accurate, context-based answers

The system consists of:

- Frontend (User Interface)
- Backend APIs
- Document Processing Pipeline
- Vector Database

- LLM API for response generation

The core concept is **RAG**, where:

- Retrieval = finding relevant document chunks
- Generation = producing answers using LLM

2. System Design

2.1 Application Design

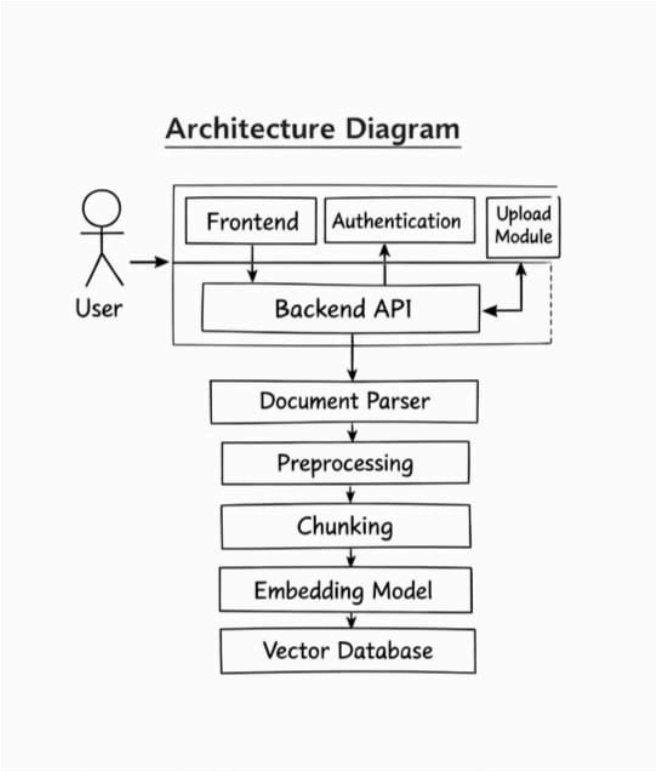
The application follows a **modular and layered architecture**:

Layers:

1. **Presentation Layer (Frontend)**
    - Handles UI/UX
    - Displays documents and chat interface
2. **Application Layer (Backend APIs)**
    - Processes requests
    - Manages logic and workflows
3. **Processing Layer**
    - Document parsing
    - Chunking
    - Embedding generation
4. **Data Layer**

- Vector Database (for embeddings)
- Relational Database (for metadata and users)

5. **External Services**
   - LLM API (for answer generation)

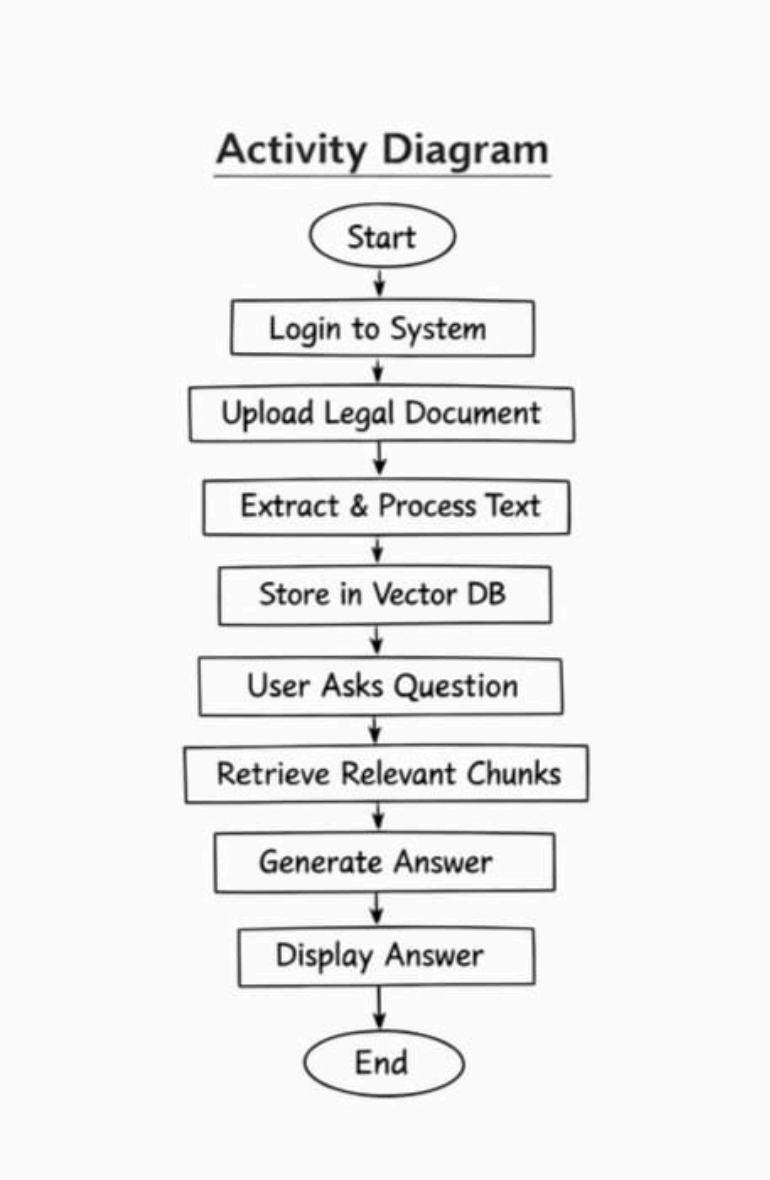


2.2 Process Flow

Document Upload Flow:

1. User uploads document
2. File validation is performed

3. Text is extracted from the document
4. Text is cleaned and preprocessed
5. Text is split into chunks
6. Embeddings are generated
7. Embeddings are stored in vector database
8. Document status is marked as "Ready"



Query Processing Flow:

1. User asks a question
2. Query is converted into embedding
3. Vector DB retrieves relevant chunks
4. Retrieved chunks + query sent to LLM
5. LLM generates contextual answer
6. Answer displayed to user
7. Chat history stored

2.3 Information Flow

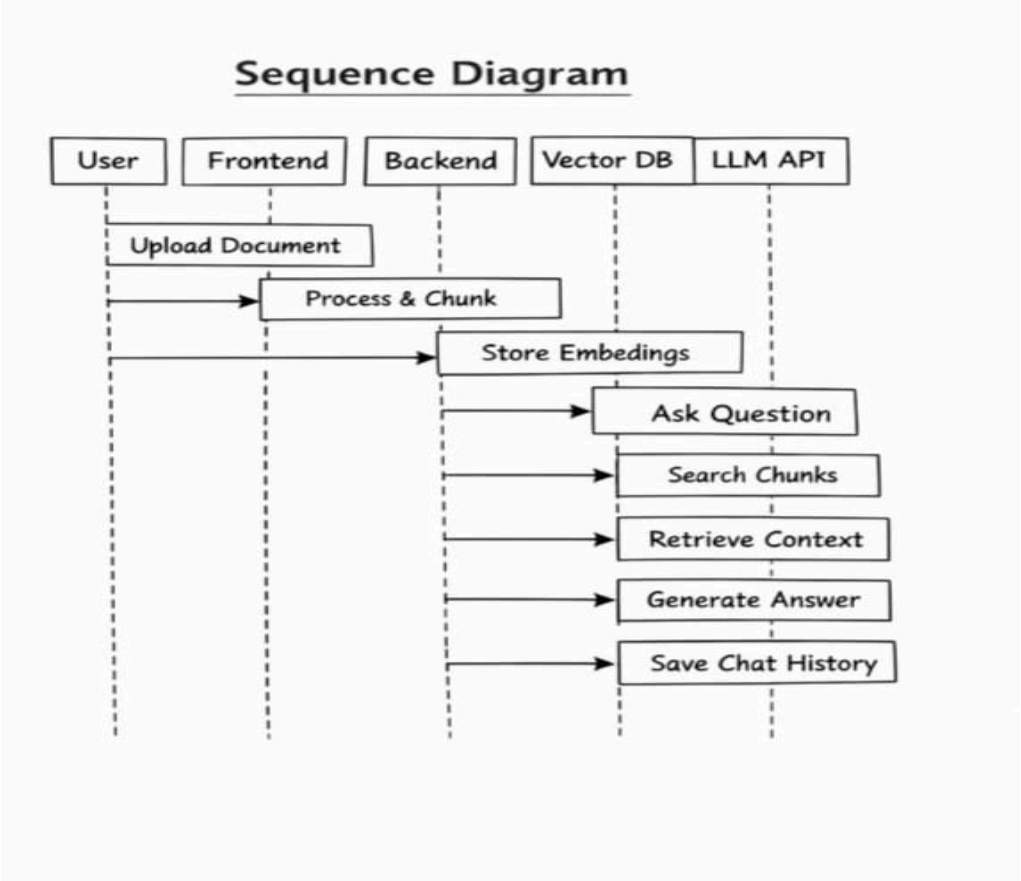
The flow of data within the system is as follows:

Upload Phase:

User → Frontend → Backend API → Document Processor → Chunking → Embedding Model → Vector DB

Query Phase:

User → Frontend → Backend API → Query Embedding → Vector DB → Retriever → LLM → Backend → Frontend



Key Data Elements:

- Document text
- Text chunks
- Embeddings (vectors)
- User queries
- Retrieved context
- Generated responses

2.4 Components Design

1. Frontend Component

- Upload interface
- Chat interface
- Dashboard
- Displays results and citations

2. Backend API Component

- Handles all requests
- Connects frontend with processing modules

3. Document Processor

- Extracts text from files
- Cleans and formats data

4. Chunking Module

- Splits text into smaller chunks
- Maintains context using overlap

5. Embedding Service

- Converts text into vector embeddings

6. Vector Database

- Stores embeddings
- Performs similarity search

7. Query Processor

- Handles user queries
- Converts queries into embeddings

8. Retriever

- Fetches relevant chunks from vector DB

9. LLM Integration Module

- Sends prompt to LLM
- Receives generated answers

10. Chat History Module

- Stores past queries and responses

2.5 Key Design Considerations

The following factors were considered during system design:

1. Accuracy

- Use of RAG ensures responses are grounded in documents

2. Scalability

- Modular design allows scaling components independently

3. Performance

- Precomputed embeddings improve query speed
- Vector DB enables fast retrieval

4. Security

- User authentication
- Secure file storage
- Access control for documents

5. Reliability

- Error handling for document processing and API calls

6. Maintainability

- Clean modular structure
- Separation of concerns

7. Usability

- Simple chat-based interface
- Easy document upload and management

---

3. References

The following resources were used in designing and implementing the system:

- OpenAI API Documentation (for embeddings and LLM)
- Hugging Face Transformers Documentation
- FAISS / Pinecone / ChromaDB Documentation
- FastAPI / Flask Documentation
- Research papers on Retrieval-Augmented Generation (RAG)